

OBSERVACIONES DE LA PRÁCTICA

1. Nicolás Fajardo Aguirre, 202413083, n.fajardoa@uniandes.edu.co

2. Christian Bravo, 202322012, ca.bravom@uniandes.edu.co

3. Dennys Lozano, 202413859, da.lozanom1@uniandes.edu.co

Ambientes de pruebas

Computador 1	
Procesador	12th Gen Intel(R) Core(TM) i5-12500H
Memoria RAM (GB)	16 gb
Sistema Operativo	Windows 11 64 bits

Tabla 1. Especificaciones del computador para ejecutar las pruebas de rendimiento.

Para realizar las siguientes pruebas, es importante que las ejecuten utilizando el archivo de datos **large**, ya que es con este conjunto donde realmente se puede observar el comportamiento y la eficiencia de los algoritmos en contextos más exigentes.

Ordenamientos Iterativos

Resultados para ordenamientos iterativos con Array List

Porcentaje de la muestra	Insertion (Array List)	Sort Selection (Array List)	Sort Shell (Array List)	Sort
0.50%	0.42	0.68	0.19	
5.00%	37.90	46.40	4.53	
10.00%	164.70	188.19	11.75	
20.00%	691.42	760.91	35.37	
30.00%	1521.43	1711.17	72.02	
50.00%	6812.23	4905.14	187.35	
80.00%	22364.24	13371.13	432.32	
100.00%	25151.83	22654.59	631.94	

Tabla 2. Resultados en computador 1 para ordenamientos iterativos con Array List.

Resultados para ordenamientos iterativos con Linked List

Porcentaje de la muestra	Insertion (Linked List)	Sort Selection (Linked List)	Sort Shell (Linked List)	Sort
0.50%	4.33	3.80	2.03	

5.00%	2965.24	3381.20	434.31
10.00%	24754.34	38877.99	2359.62
20.00%	202321.73	362774.97	27320.11
30.00%			78028.06
50.00%			254006.03
80.00%			
100.00%			

Tabla 3. Resultados en computador 1 para ordenamientos iterativos con Single Linked List.

Ordenamientos Recursivos

Resultados para ordenamientos recursivos con Array List

Porcentaje de muestra	de la Merge (Array List)	Sort Quick (Array List)	Sort
0.50%	0.24	0.34	
5.00%	2.72	3.62	
10.00%	5.83	9.99	
20.00%	13.90	26.40	
30.00%	20.46	53.48	
50.00%	37.94	116.21	
80.00%	58.64	297.78	
100.00%	75.32	391.95	

Tabla 4. Resultados en computador 1 para ordenamientos recursivos con Array List.

Resultados para ordenamientos recursivos con Linked List

Porcentaje de muestra	de la Merge (Linked List)	Sort Quick (Linked List)	Sort
0.50%	0.51	1.10	
5.00%	20.84	38.39	
10.00%	73.49	252.06	
20.00%	301.97	867.92	
30.00%	705.43	1555.20	
50.00%	1878.21	5260.82	
80.00%	4866.72	15920.20	
100.00%	9361.26	65943.46	

Tabla 5. Resultados en computador 1 para ordenamientos recursivos con Single Linked List.

Una vez alla llenado l

Comparación de tiempo mejores algoritmos de ordenamiento

Complete esta sección únicamente cuando se le indique en las instrucciones. En este punto, deberá comparar el tiempo de ejecución entre el mejor algoritmo de ordenamiento recursivo y el mejor algoritmo de ordenamiento iterativo, los cuales debió haber identificado en el punto anterior.

Por favor, registre las mediciones en el espacio correspondiente de la tabla, especificando claramente:

- El nombre de cada algoritmo utilizado
- El tipo de estructura de datos empleada (*ArrayList* o *SingleLinkedList*)

Asegúrese de completar todos los campos solicitados en la tabla e incluir los nombres según corresponda, no es necesario que vuelva a correr las pruebas esta información la puede sacar de las tablas anteriores.

Computador 1

Porcentaje de la muestra	Algoritmo recursivo (Merge Sort - ArrayList)	Algoritmo iterativo (Shell Sort - ArrayList)
0.50%	0.24	0.19
5.00%	2.72	4.53
10.00%	5.83	11.75
20.00%	13.9	35.37
30.00%	20.46	72.02
50.00%	37.94	187.35
80.00%	58.64	432.32
100.00%	75.32	631.94

Tabla 6. Resultados en computador 1 mejores algoritmos

Preguntas de análisis parte 1

1. ¿Cómo varía el comportamiento de cada algoritmo con respecto al tamaño de los datos?

Para responder, agregue líneas de tendencia en las gráficas de las pestañas 04 a 08. Observe si el crecimiento del tiempo de ejecución es lineal, cuadrático o de otro tipo, y relacione ese patrón con la complejidad teórica esperada de cada algoritmo.

RTA: En general, cada algoritmo respeta su complejidad teórica. Insertion Sort y Selection Sort muestran un crecimiento cuadrático ($O(n^2)$): al duplicar el tamaño de la muestra, el tiempo se multiplica entre 3.7x y 4.6x, cercano al 4x esperado. Shell Sort crece más lento (3.4x al duplicar n), consistente con su complejidad intermedia ($O(n^{1.3})$). Merge Sort es

el que mejor sigue su complejidad teórica $O(n \log n)$: al duplicar el tamaño, el tiempo apenas se duplica (2.0x). Quick Sort, en cambio, creció más de lo esperado para $O(n \log n)$ (3.4x al duplicar n), posiblemente porque la estrategia de partición usada no siempre genera particiones balanceadas.

2. **¿Qué diferencias observas en el rendimiento de un mismo algoritmo al utilizar ArrayList frente a SingleLinkedList?**

Analice esta comparación en las pestañas 04 a 08, donde se muestran ambos casos. Justifique su respuesta con base en las líneas de tendencia.

RTA: La diferencia es muy marcada: todos los algoritmos son considerablemente más lentos en SingleLinkedList que en ArrayList, y en varios casos cambia la complejidad observada, no solo la constante. Insertion Sort y Selection Sort en SingleLinkedList crecen 8x-9x al duplicar el tamaño de la muestra (comportamiento cúbico), mientras que en ArrayList crecen de forma cuadrática. Merge Sort pasa de $O(n \log n)$ en ArrayList a un crecimiento cercano a $O(n^2)$ en la lista encadenada. Esto se debe a que `get_element()` y `exchange()` en `single_linked_list.py` son $O(n)$ (recorren la lista desde el primer nodo en cada llamado), mientras que en `array_list.py` son $O(1)$ por acceso directo al índice. Como los algoritmos de ordenamiento llaman estas funciones repetidamente dentro de sus ciclos, esa ineficiencia se multiplica por cada invocación, degradando fuertemente el rendimiento en listas encadenadas.

3. **¿Qué algoritmo iterativo mostró el mejor comportamiento general en términos de tiempo y cantidad de datos en ambas estructuras de datos?**

Justifique su elección comparando las gráficas (04 Insertion Sort), (05 Selection Sort) y (06 Shell Sort) y apoyándose en las gráficas de las pestañas 2 y 3.

RTA: Shell Sort mostró el mejor comportamiento general entre los algoritmos iterativos, en ambas estructuras. En ArrayList, a 10,000 elementos, tardó 631.94 ms frente a 22,364.24 ms de Insertion Sort y 22,654.59 ms de Selection Sort (más de 30 veces más rápido). En SingleLinkedList fue el único de los tres iterativos que logró completarse hasta el 50% de la muestra dentro de un tiempo razonable; Insertion Sort y Selection Sort no pudieron completarse más allá del 20% porque superaban los 10 minutos de ejecución. Esto concuerda con su complejidad teórica ($O(n^{1.3})$), notablemente menor que la de Insertion y Selection Sort ($O(n^2)$).

4. **¿Qué algoritmo recursivo presentó el mejor desempeño general según los tiempos registrados y la forma de la línea de tendencia?**

Justifique su respuesta basándose en las pestañas 07 (Merge Sort) y 08 (Quick Sort) y apoyándose en las gráficas de las pestañas 2 y 3.

RTA: Merge Sort presentó el mejor desempeño general entre los algoritmos recursivos, en ambas estructuras. En ArrayList, a 10,000 elementos, tardó 75.32 ms frente a 391.95 ms

de Quick Sort (5 veces más rápido). La diferencia es aún mayor en SingleLinkedList: 9,361.26 ms frente a 65,943.46 ms de Quick Sort. Esto concuerda con la forma de las líneas de tendencia: Merge Sort mantiene un crecimiento cercano a $O(n \log n)$ incluso en la lista encadenada, mientras que Quick Sort se ve más afectado por el costo de acceso $O(n)$, en parte porque su estrategia de partición no garantiza subdivisiones balanceadas.

5. **Con base en su análisis visual y los datos recolectados, ¿cuáles algoritmos seleccionan como los “mejores” en cada categoría (iterativo y recursivo)?**

Indique también la estructura de datos en la que se desempeñó mejor cada uno.

RTA: El mejor algoritmo iterativo es Shell Sort y el mejor algoritmo recursivo es Merge Sort, ambos con mejor desempeño en ArrayList que en SingleLinkedList. En ArrayList, a 10,000 elementos, Shell Sort tardó 631.94 ms y Merge Sort 75.32 ms. En SingleLinkedList, Shell Sort solo logró completarse hasta 5,000 elementos (254,006.03 ms) antes de superar el límite de tiempo, mientras que Merge Sort sí completó los 10,000 elementos (9,361.26 ms). Considerando ambas estructuras, Merge Sort es el algoritmo con el mejor desempeño general.

Preguntas de análisis parte 2

1. **¿Cuál de los dos algoritmos seleccionados como “mejores” muestra un comportamiento más eficiente a medida que crece el tamaño de la muestra?**

Para responder, agregue líneas de tendencia en la gráfica de la pestaña 09-Bests, identifique cuál algoritmo crece más lentamente y relacione ese patrón con su complejidad teórica. Use esta comparación para justificar cuál es más escalable.

RTA: Merge Sort mostró el comportamiento más eficiente y escalable a medida que crece el tamaño de la muestra. Su tiempo se multiplica por apenas 2.0x al duplicar el tamaño (de 5,000 a 10,000: de 37.94 ms a 75.32 ms), consistente con su complejidad $O(n \log n)$. Shell Sort, en cambio, se multiplica por $\sim 3.4x$ en el mismo rango (de 187.35 ms a 631.94 ms), reflejando su complejidad teórica $O(n^{1.3})$, superior a la de Merge Sort. La línea de tendencia de Merge Sort en la pestaña 09-Bests es visiblemente más plana, y la brecha entre ambos algoritmos se agranda a medida que crece n , confirmando que Merge Sort es más escalable para conjuntos de datos grandes.

2. **¿Qué estructura de datos resultó más favorable para cada algoritmo seleccionado como mejor?**

Analice si la ventaja se mantiene constante en todos los tamaños o solo en ciertos rangos.

RTA: En ambos casos ArrayList resultó más favorable, y esa ventaja se mantiene en todos los tamaños probados (no hay ningún tamaño donde SingleLinkedList sea más rápida), pero la ventaja no es constante: crece con el tamaño de la muestra. Para Shell

Sort, la razón de tiempos SingleLinkedList/ArrayList pasa de 11x en $n=50$ a más de 1,300x en $n=5,000$. Para Merge Sort la brecha también crece, pero de forma más moderada: de 2x en $n=50$ a $\sim 124x$ en $n=10,000$. Esto confirma que el costo de acceso $O(n)$ de SingleLinkedList afecta más severamente a Shell Sort que a Merge Sort, ya que Shell Sort realiza muchos más accesos individuales por elemento.

3. **¿Qué ventajas prácticas podría tener implementar el algoritmo ganador en aplicaciones reales donde se manejan grandes volúmenes de datos?**

RTA: Merge Sort sería la mejor opción para aplicaciones que manejan grandes volúmenes de datos porque su complejidad $O(n \log n)$ garantiza un crecimiento predecible del tiempo de ejecución incluso cuando el conjunto de datos crece considerablemente, a diferencia de algoritmos con complejidad cuadrática o peor. Además, al no depender de la elección de un pivote (a diferencia de Quick Sort), su rendimiento es consistente sin casos patológicos de peor caso. Esto lo hace ideal para sistemas que procesan catálogos grandes, como el de este laboratorio con miles de libros, donde se requiere ordenar datos de forma periódica y predecible, por ejemplo en motores de búsqueda o generación de reportes.

4. **Si se presentara una lista parcialmente ordenada, ¿cambiaría su elección de algoritmo? Explique por qué, con base en el comportamiento observado.**

RTA: Sí, cambiaría la elección en algunos casos. Insertion Sort es un algoritmo adaptativo: su complejidad se reduce a $O(n)$ cuando los datos están casi ordenados, porque cada elemento requiere pocos o ningún desplazamiento. Merge Sort, en cambio, siempre realiza $O(n \log n)$ comparaciones sin importar el orden inicial de los datos, ya que su estrategia de dividir y mezclar no aprovecha el orden preexistente. Por lo tanto, si se supiera de antemano que la lista está parcialmente ordenada, Insertion Sort podría ser más eficiente en la práctica que Merge Sort, a pesar de tener una complejidad de peor caso más alta. Sin embargo, en SingleLinkedList esto no aplicaría de la misma manera, porque el costo $O(n)$ de acceso a cada elemento seguiría penalizando fuertemente a Insertion Sort, como se observó en nuestras pruebas.